

Department of Computing and Mathematics
ASSESSMENT COVER SHEET

Unit code and title:	6G4Z0024 - Web Development
Assignment set by:	Dr. Yanlong Zhang
Assignment ID:	1CWK100
Assignment weighting:	100%
Assignment title:	A structured website
Type: (Group/Individual)	Individual
Hand-in deadline:	15 May 2026
Hand-in format and mechanism:	Your work must be submitted through Moodle. The deliverable is ONE zipped file containing: • All your code to run the website • A Word/PDF document for the report

Learning Outcomes Being Assessed:

- LO1:** Create basic web pages using HTML to mark-up content, using CSS to control its presentation in web browsers
- LO2:** Write efficient and readable client-side scripts that are event- and object-driven and runs on multiple browsers and platform
- LO3:** Apply web design usability principles in the creation of web content based on the requirements of a given scenario

Note: It is your responsibility to make sure that your work is complete and available for marking by the deadline. Make sure that you have followed the submission instructions carefully, and that your work is submitted in the correct format, using the correct hand-in mechanism (e.g., Moodle upload). If submitting via Moodle, you are advised to check your work after upload, to make sure it has uploaded properly. **Do not alter your work after the deadline.** You should make at least one full backup copy of your work.

The Timeliness of Submissions is Strictly Monitored and Enforced.

Please note that individual tutors are unable to grant extensions to coursework. Extensions can only be granted based on approved Exceptional Factors.

Plagiarism: The students will follow Manchester Met policy for plagiarism. For further information on the Manchester Met policy, please see [Academic Integrity and Misconduct](#).

Plagiarism is the unacknowledged representation of another person's work, or use of their ideas, as one's own. Manchester Metropolitan University takes care to detect plagiarism, employs plagiarism detection software, and imposes severe penalties, as outlined in the Student Handbook. If in doubt, seek advice from your tutor.

Your work will be submitted through Moodle.

Assessment Criteria:	As indicated in the attached assignment specification
Formative Feedback:	Lecture/Lab discussion and interactive with tutor onwards from when the assignment is set.
Summative Feedback Format:	You will be given individual feedback via Moodle.

Introduction

This coursework will consolidate your position on everything you have met over the block during the Web Development unit.

A web site should be created that mimics the operation of the following pages in a “payment gateway” of an e-commerce website. In broad terms, marks are awarded for the extent to which the page you create emulates the payment gateway acquisition of credit card details and validates those details using Regular Expressions.

Aim

This unit encourages you to analyse real-world situations critically. The assessment mimics industry projects by requiring you to engage with multiple disciplines. By the end of the unit, you will have completed the development of an application that uses a variety of web technologies. It is encouraged that you maintain a portfolio of projects throughout university (e.g. through GitHub) that can serve as a portfolio of your work when applying for jobs. This project could serve as one aspect of your portfolio. The following skills will be essential for successful completion of this coursework (and including such a project in your portfolio would demonstrate these skills to potential employers):

- **Real-World Problem Solving:** You will need to analyse a real-world situation, develop solutions for multiple problems when developing the application, and then evaluate your solutions.
- **Technical Skills:** This assessment requires you to write an application using the fundamental technologies that make up the web. In addition to these technologies, you will gain a foundational understanding of how the web works.

The Assessment (1CWK100)

You are required to develop the following:

- A. Page 1, Homepage, index.html:** A page to offer site navigation and a list of books to sell. You are required to include a navigation bar, consisting of three elements, ‘**Home**’, ‘**About Us**’, and ‘**Contact Us**’. You are not required to develop ‘About Us’ and ‘Contact Us’ pages.

Choose your favourite books (at least four), each book must contain two columns, Column 1) face image, Column 2) a short introduction and a ‘Pay’ button.

Responsive Design: Your book list should be in two-column format on desktop view, but single column on smartphones, see examples in Figure 1 and Figure 2.

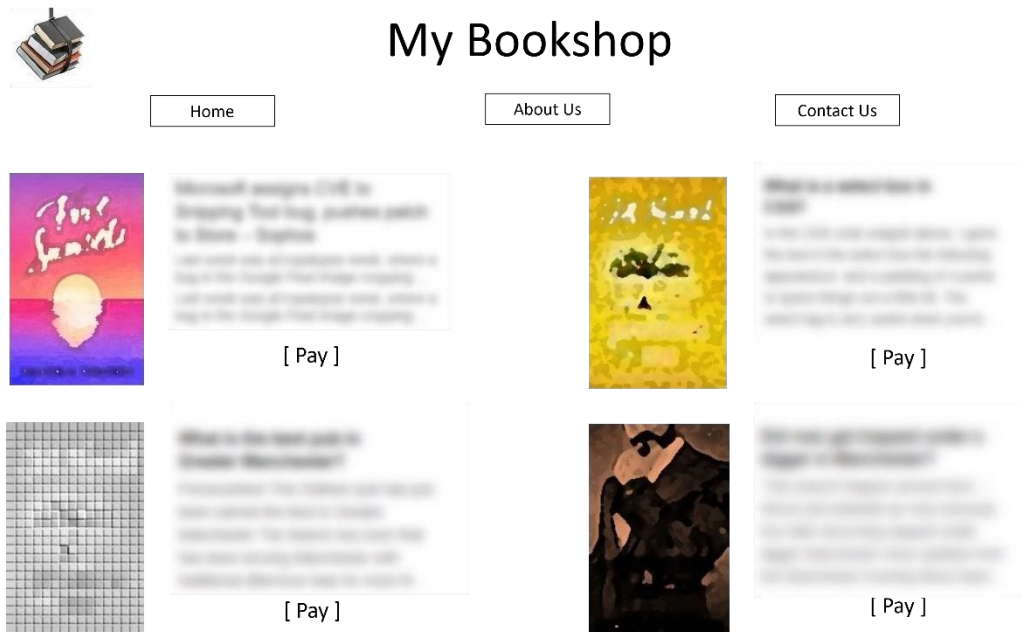


Figure 1. an example of homepage in Desktop View

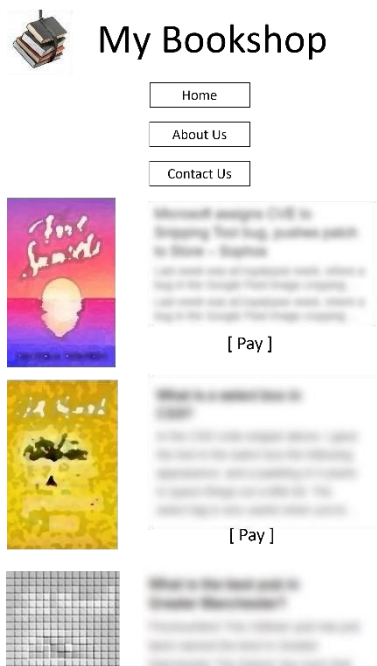


Figure 2. An example of homepage in smartphone view

Please note: You will be rewarded extra marks for responsive nav design, like that in Figure 3. (you will learn this in the Lab session).

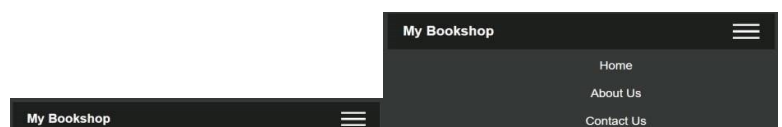


Figure 3. A professional way for nav design in smartphone view

B. Page 2, pay.html: A page to accept user's credit card details. An example is shown in Figure 4.

The intention behind the web page is that a user will put in their credit card details. The user will then click on "**Continue**" and the credit card number (in this case, "Mastercard") will be checked by a JavaScript code. Amongst other things this JavaScript code will check (1) the total number of entered digits for credit cards is 16, and (2) the MasterCard number starting with 51, 52, 53, 54, or 55, (3) the card is not expired, (4) the total number of digits for security code is 3 or 4. If the JavaScript check is unsatisfied, users are asked to re-fill in the form. On passing validation, use JavaScript to send a POST request to the server (<https://mudfoot.doc.stu.mmu.ac.uk/node/api/creditcard>). The server expects raw JSON to be sent in the following format:

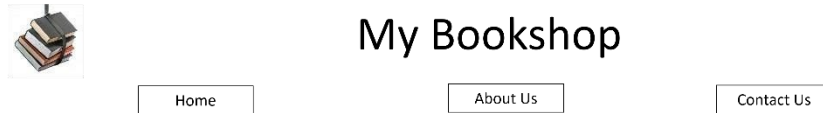
```
{
    "master_card": 5222123456781234,
    "exp_year": 2025,
    "exp_month": 11,
    "cvv_code": "089"
}
```

Make sure that your JavaScript can handle errors coming back from the server (e.g., if the wrong data is sent). This is another layer of validation.

On success (200 OK from the server), parse the response and provide a success message to the user.

```
{
    "message": "Thank you for your payment."
}
```

Please Note: you must complete the card checking using JavaScript. No mark is awarded for using other programming languages.



Payment Options

Debit / Credit Card



Card Number

Expiration Date

Security Code

(3-4 digit code normally on the back of your card)

Figure 4. an example of pay.html in desktop view

C. Page 3, success.html: A page to offer feedback if the payment is successful. An example is shown in Figure 5.

Please note that 'success.html' also returns the last four digits of the credit card number which had been input by the user, in the form "**** * 1234".

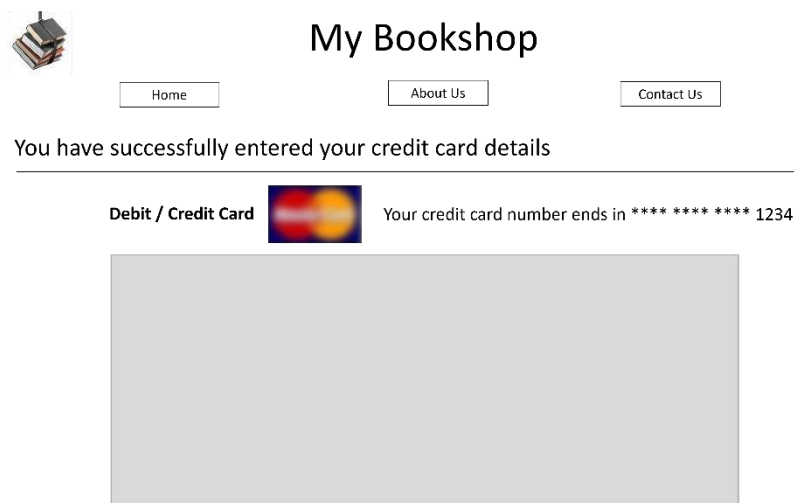


Figure 5. an example of success.html in desktop view

D. additional challenge:

You may extend the JavaScript functionality to support purchasing multiple books rather than a single item. This may include allowing users to select more than one book, dynamically calculating and displaying the total price before payment, and passing this information between pages using JavaScript (e.g. via sessionStorage). The total price should be clearly presented to the user on the payment page and/or success page. This feature is optional and

WEB DEVELOPMENT: Assessment Specification

should be implemented using client-side JavaScript only, without changing the core payment validation requirements.

CSS requirements

You must create **one external CSS file** stored in the css folder and link it to **all HTML pages** (index.html, pay.html, and success.html). Your CSS should control the overall layout, typography, colours, spacing, and visual consistency of the website. You are expected to demonstrate **responsive design**, ensuring the site works well on both desktop and smartphone screens (e.g. multi-column layout on desktop and single-column layout on mobile). Use appropriate CSS techniques such as flexbox or media queries. **You must not use Bootstrap or any other CSS frameworks or third-party libraries**; all styling must be written by you using plain (vanilla) CSS.

The following directory structure is to be used for all the files of your website: root
folder *index.html pay.html success.html*

- js folder

All JavaScript codes

-image folder

All images

-css folder

A single CSS file

The directory structure that you will be using is a little “old school”. A more modern approach is to use an MVC framework. The MVC methodology is beyond the scope of the current course although it is alluded to during the lectures. Notwithstanding this observation, WordPress (the most used web site builder on the web) uses a derivative of the above directory structure. Please adhere to the above directory structure i.e NOT MVC.

Submission Instructions

This should be done over Moodle and is ONE zipped (compressed) file of two components:

- 1] a report, see the specification below.
- 2] all the JavaScript, HTML, and CSS code needed to operate the website in the appropriate directories. To demonstrate your understanding of the JavaScript code, you must include the required comments (see to the lecture notes for details on comment requirements).

The Report

Usability and Accessibility Checklist

Implement features to address usability and accessibility. List your features, explaining briefly how each benefits usability and/or accessibility.

Please refer to Lecture 4b Website Usability and Accessibility

Marking Grid

Report [10%]	List at least 5 points, 2 marks each.
Software [90%]	See below.

Assessment Marking Criteria

	Fail (0 to 29%)	Marginal Fail (30 to 39%)	3 rd Class (40 to 49%)	2 nd Class: 2 (50 to 59%)	2 nd Class: 1 (60 to 69%)	1 st Class (70 to 85%)	Exceptional 1 st (86 to 100%)
HTML 20%	Little or no HTML has been submitted.	You have submitted some HTML, but it is not well formed and littered with errors.	You have submitted at least one HTML document that parses correctly on the screen. However, it is lacking in functionality and/or contains some error	Your HTML is mostly well formed and covers most elements required. However, other elements are missing and/or you have consistently used inappropriate elements.	You HTML is well-formed and works. However, there are some examples of errors and/or inappropriate element use scattered throughout.	You have submitted multiple well-formed HTML pages that link together. You have covered all elements required but there are parts of your site where other elements would have been appropriate.	You have submitted multiple well-formed HTML pages that link together and contain all elements required in a way that is appropriate.
CSS 30%	Little or no CSS has been submitted.	You have submitted some CSS, but it is not well formed and littered with errors.	You have an external CSS sheet, which is correctly pulled into at least one HTML file. However, it is lacking in functionality and/or contains some errors.	Your CSS is stored in an external sheet and linked to by all HTML files. Your styles cover a large portion of the site, but you haven't fully demonstrated what the specification asks.	Your external CSS demonstrates the points asked for in the specification. Site is responsive with errors. However, there are some examples of errors and/or areas for improvement.	Your external CSS sheet contains styling for the entire site. You have covered all points in the specification, Site is fully responsive, but there are parts where you could have made your CSS more efficient.	Your external CSS sheet contains styling for the entire site to a professional standard. Site is working well in desktop and smartphone views.

JavaScript 40%	Little or no attempt has been made.	You have a HTML form that calls JS validation functions on submission. However, they don't work/are littered with errors.	You have appropriate validation functions that prevent the form from submitting. You have attempted the API request but there are a lot of errors	Your validation and API call mostly works. However, you are incorrectly handling the call/response and/or you aren't following the specification. There are also some bugs throughout	The validation and API call works and mostly fits with the requirements outlined in the specification. There are a few bugs and/or areas that could be made more efficient.	Your validation and API call works and adheres to the specification. Success page mostly works but there are some minor bugs (such as last four card number shown).	Your validation and API call works, and the success page follows the specification. Additional challenge completed successfully. Complete and correct comments.
--	--	--	--	--	--	--	--

